

Team members:

Menna Mahgoub 231000843
Farah Tarek 241002520
Sandy Saad 241001673
Rawan Mamdouh 241001796
Malak amr ashour 221001703



UI Application model

The UI code used:

```
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector

# --- DB Connection ---
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "ABCD@1234",
    "database": "ClinicManagement"
}

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)

# --- Main App ---
class ClinicApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Clinic Management System")
        self.geometry("1000x650")
        self.configure(bg="#f0f4f8")
        self.resizable(True, True)

        header = tk.Frame(self, bg="#1a73e8", height=60)
        header.pack(fill="x")
        tk.Label(header, text=" Clinic Management System",
            font=("Segoe UI", 18, "bold"), bg="#1a73e8", fg="white").pack(side="left", padx=20, pady=10)

        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TNotebook", background="#f0f4f8", borderwidth=0)
        style.configure("TNotebook.Tab", font=("Segoe UI", 11), padding=[14, 6])
        style.map("TNotebook.Tab", background=[("selected", "#1a73e8")], foreground=[("selected", "white")])

        nb = ttk.Notebook(self)
        nb.pack(fill="both", expand=True, padx=10, pady=10)

        nb.add(DepartmentTab(nb), text=" Departments")
        nb.add(ClinicTab(nb), text=" Clinics")
        nb.add(DoctorTab(nb), text=" Doctors")
        nb.add(PatientTab(nb), text=" Patients")
        nb.add(AppointmentTab(nb), text=" Appointments")
        nb.add(QueryTab(nb), text=" Queries")

    def make_form_row(parent, row, label, widget_factory):
        tk.Label(parent, text=label, font=("Segoe UI", 10), bg="white", anchor="w").grid(
            row=row, column=0, sticky="w", padx=(10, 5), pady=4)
        w = widget_factory(parent)
        w.grid(row=row, column=1, sticky="ew", padx=(0, 10), pady=4)
        return w

    def make_entry(parent):
        return tk.Entry(parent, font=("Segoe UI", 10), relief="solid", bd=1)

    def build_tree(parent, columns):
        frame = tk.Frame(parent, bg="white")
        frame.pack(fill="both", expand=True, padx=10, pady=(0, 10))
        tree = ttk.Treeview(frame, columns=columns, show="headings", height=12)
        for c in columns:
            tree.heading(c, text=c)
            tree.column(c, width=120, anchor="center")
```

```

sb = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
tree.configure(yscrollcommand=sb.set)
tree.pack(side="left", fill="both", expand=True)
sb.pack(side="right", fill="y")
return tree

def action_btn(parent, text, color, cmd):
    return tk.Button(parent, text=text, font=("Segoe UI", 10, "bold"),
        bg=color, fg="white", relief="flat", cursor="hand2",
        padx=14, pady=6, command=cmd)

# === Department ===
class DepartmentTab(tk.Frame):
    def __init__(self, parent):
        super().__init__(parent, bg="ffffff")
        self._build()

    def _build(self):
        form = tk.LabelFrame(self, text=" Add Department ", font=("Segoe UI", 10, "bold"),
            bg="ffffff", fg="#1a73e8", relief="groove", bd=2)
        form.pack(fill="x", padx=10, pady=10)
        form.columnconfigure(1, weight=1)

        self.dept_id = make_form_row(form, 0, "Department ID:", make_entry)
        self.dept_name = make_form_row(form, 1, "Department Name:", make_entry)

        btn_frame = tk.Frame(form, bg="ffffff")
        btn_frame.grid(row=2, column=0, columnspan=2, pady=8)
        action_btn(btn_frame, "+ Add", "#1a73e8", self.add).pack(side="left", padx=5)
        action_btn(btn_frame, "Delete", "#e53935", self.delete).pack(side="left", padx=5)
        action_btn(btn_frame, "Refresh", "#43a047", self.load).pack(side="left", padx=5)

        self.tree = build_tree(self, ["DepartmentID", "DepartmentName"])
        self.load()

    def load(self):
        for r in self.tree.get_children(): self.tree.delete(r)
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("SELECT DepartmentID, DepartmentName FROM Department")
            for row in cur.fetchall(): self.tree.insert("", "end", values=row)
            con.close()
        except Exception as e:
            messagebox.showerror("Error", str(e))

    def add(self):
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("INSERT INTO Department (DepartmentID, DepartmentName) VALUES (%s, %s)",
                (self.dept_id.get(), self.dept_name.get()))
            con.commit(); con.close()
            self.dept_id.delete(0, "end"); self.dept_name.delete(0, "end")
            self.load()
            messagebox.showinfo("Success", "Department added!")
        except Exception as e:
            messagebox.showerror("Error", str(e))

    def delete(self):
        sel = self.tree.selection()
        if not sel: return messagebox.showwarning("Warning", "Select a row first!")
        val = self.tree.item(sel[0])["values"][0]
        if messagebox.askyesno("Confirm", f"Delete Department {val}?"):
            try:
                con = get_connection(); cur = con.cursor()
                cur.execute("DELETE FROM Department WHERE DepartmentID=%s", (val,))
                con.commit(); con.close(); self.load()
            except Exception as e:
                messagebox.showerror("Error", str(e))

# === Clinic ===
class ClinicTab(tk.Frame):
    def __init__(self, parent):
        super().__init__(parent, bg="ffffff")
        self._build()

    def _build(self):
        form = tk.LabelFrame(self, text=" Add Clinic ", font=("Segoe UI", 10, "bold"),
            bg="ffffff", fg="#1a73e8", relief="groove", bd=2)
        form.pack(fill="x", padx=10, pady=10)
        form.columnconfigure(1, weight=1)

        self.clinic_id = make_form_row(form, 0, "Clinic ID:", make_entry)
        self.clinic_name = make_form_row(form, 1, "Clinic Name:", make_entry)
        self.address = make_form_row(form, 2, "Address:", make_entry)
        self.dept_id = make_form_row(form, 3, "Department ID:", make_entry)

        btn_frame = tk.Frame(form, bg="ffffff")

```

```

btn_frame.grid(row=4, column=0, columnspan=2, pady=8)
action_btn(btn_frame, "➕ Add", "#1a73e8", self.add).pack(side="left", padx=5)
action_btn(btn_frame, "🗑 Delete", "#e53935", self.delete).pack(side="left", padx=5)
action_btn(btn_frame, "🔄 Refresh", "#43a047", self.load).pack(side="left", padx=5)

self.tree = build_tree(self, ["ClinicID", "ClinicName", "Address", "DepartmentID"])
self.load()

def load(self):
    for r in self.tree.get_children(): self.tree.delete(r)
    try:
        con = get_connection(); cur = con.cursor()
        cur.execute("SELECT ClinicID, ClinicName, Address, DepartmentID FROM Clinic")
        for row in cur.fetchall(): self.tree.insert("", "end", values=row)
        con.close()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def add(self):
    try:
        con = get_connection(); cur = con.cursor()
        cur.execute("INSERT INTO Clinic (ClinicID, ClinicName, Address, DepartmentID) VALUES (%s, %s, %s, %s)",
            (self.clinic_id.get(), self.clinic_name.get(),
             self.address.get(), self.dept_id.get()))
        con.commit(); con.close()
        for f in [self.clinic_id, self.clinic_name, self.address, self.dept_id]:
            f.delete(0, "end")
        self.load()
        messagebox.showinfo("Success", "Clinic added!")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def delete(self):
    sel = self.tree.selection()
    if not sel: return messagebox.showwarning("Warning", "Select a row first!")
    val = self.tree.item(sel[0])["values"][0]
    if messagebox.askyesno("Confirm", f"Delete Clinic {val}?"):
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("DELETE FROM Clinic WHERE ClinicID=%s", (val,))
            con.commit(); con.close(); self.load()
        except Exception as e:
            messagebox.showerror("Error", str(e))

# ===== Doctor =====
class DoctorTab(tk.Frame):
    def __init__(self, parent):
        super().__init__(parent, bg="#ffffff")
        self._build()

    def _build(self):
        form = tk.LabelFrame(self, text=" Add Doctor ", font=("Segoe UI", 10, "bold"),
            bg="#ffffff", fg="#1a73e8", relief="groove", bd=2)
        form.pack(fill="x", padx=10, pady=10)
        form.columnconfigure(1, weight=1)

        self.doc_id = make_form_row(form, 0, "Doctor ID:", make_entry)
        self.doc_name = make_form_row(form, 1, "Name:", make_entry)
        self.phone = make_form_row(form, 2, "Phone:", make_entry)
        self.address = make_form_row(form, 3, "Address:", make_entry)
        self.dept_id = make_form_row(form, 4, "Department ID:", make_entry)

        btn_frame = tk.Frame(form, bg="#ffffff")
        btn_frame.grid(row=5, column=0, columnspan=2, pady=8)
        action_btn(btn_frame, "➕ Add", "#1a73e8", self.add).pack(side="left", padx=5)
        action_btn(btn_frame, "🗑 Delete", "#e53935", self.delete).pack(side="left", padx=5)
        action_btn(btn_frame, "🔄 Refresh", "#43a047", self.load).pack(side="left", padx=5)

        self.tree = build_tree(self, ["DoctorID", "DoctorName", "PhoneNumber", "Address", "DepartmentID"])
        self.load()

    def load(self):
        for r in self.tree.get_children(): self.tree.delete(r)
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("SELECT DoctorID, DoctorName, PhoneNumber, Address, DepartmentID FROM Doctor")
            for row in cur.fetchall(): self.tree.insert("", "end", values=row)
            con.close()
        except Exception as e:
            messagebox.showerror("Error", str(e))

    def add(self):
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("INSERT INTO Doctor (DoctorID, DoctorName, PhoneNumber, Address, DepartmentID) VALUES (%s, %s, %s, %s, %s)",
                (self.doc_id.get(), self.doc_name.get(), self.phone.get(),
                 self.address.get(), self.dept_id.get()))

```

```

        con.commit(); con.close()
    for f in [self.doc_id, self.doc_name, self.phone, self.address, self.dept_id]:
        f.delete(0, "end")
    self.load()
    messagebox.showinfo("Success", "Doctor added!")
except Exception as e:
    messagebox.showerror("Error", str(e))

def delete(self):
    sel = self.tree.selection()
    if not sel: return messagebox.showwarning("Warning", "Select a row first!")
    val = self.tree.item(sel[0])["values"][0]
    if messagebox.askyesno("Confirm", f"Delete Doctor {val}?"):
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("DELETE FROM Doctor WHERE DoctorID=%s", (val,))
            con.commit(); con.close(); self.load()
        except Exception as e:
            messagebox.showerror("Error", str(e))

# === Patient ===
class PatientTab(tk.Frame):
    def __init__(self, parent):
        super().__init__(parent, bg="#ffffff")
        self._build()

    def _build(self):
        form = tk.LabelFrame(self, text=" Add Patient ", font=("Segoe UI", 10, "bold"),
                             bg="#ffffff", fg="#1a73e8", relief="groove", bd=2)
        form.pack(fill="x", padx=10, pady=10)
        form.columnconfigure(1, weight=1)

        self.pat_id = make_form_row(form, 0, "Patient ID:", make_entry)
        self.pat_name = make_form_row(form, 1, "Name:", make_entry)
        self.phone = make_form_row(form, 2, "Phone:", make_entry)
        self.address = make_form_row(form, 3, "Address:", make_entry)
        self.birthdate = make_form_row(form, 4, "Birth Date (YYYY-MM-DD):", make_entry)
        self.job = make_form_row(form, 5, "Job:", make_entry)

        btn_frame = tk.Frame(form, bg="#ffffff")
        btn_frame.grid(row=6, column=0, columnspan=2, pady=8)
        action_btn(btn_frame, "➕ Add", "#1a73e8", self.add).pack(side="left", padx=5)
        action_btn(btn_frame, "🗑 Delete", "#e53935", self.delete).pack(side="left", padx=5)
        action_btn(btn_frame, "🔄 Refresh", "#43a047", self.load).pack(side="left", padx=5)

        self.tree = build_tree(self, ["PatientID", "PatientName", "PhoneNumber", "Address", "BirthDate", "Job"])
        self.load()

    def load(self):
        for r in self.tree.get_children(): self.tree.delete(r)
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("SELECT PatientID, PatientName, PhoneNumber, Address, BirthDate, Job FROM Patient")
            for row in cur.fetchall(): self.tree.insert("", "end", values=row)
            con.close()
        except Exception as e:
            messagebox.showerror("Error", str(e))

    def add(self):
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("INSERT INTO Patient (PatientID, PatientName, PhoneNumber, Address, BirthDate, Job) VALUES (%s, %s, %s, %s, %s, %s)",
                        (self.pat_id.get(), self.pat_name.get(), self.phone.get(),
                         self.address.get(), self.birthdate.get(), self.job.get()))
            con.commit(); con.close()
            for f in [self.pat_id, self.pat_name, self.phone, self.address, self.birthdate, self.job]:
                f.delete(0, "end")
            self.load()
            messagebox.showinfo("Success", "Patient added!")
        except Exception as e:
            messagebox.showerror("Error", str(e))

    def delete(self):
        sel = self.tree.selection()
        if not sel: return messagebox.showwarning("Warning", "Select a row first!")
        val = self.tree.item(sel[0])["values"][0]
        if messagebox.askyesno("Confirm", f"Delete Patient {val}?"):
            try:
                con = get_connection(); cur = con.cursor()
                cur.execute("DELETE FROM Patient WHERE PatientID=%s", (val,))
                con.commit(); con.close(); self.load()
            except Exception as e:
                messagebox.showerror("Error", str(e))

# === Appointment ===
class AppointmentTab(tk.Frame):

```

```

def __init__(self, parent):
    super().__init__(parent, bg="#ffffff")
    self._build()

def _build(self):
    form = tk.LabelFrame(self, text=" Add Appointment ", font=("Segoe UI", 10, "bold"),
        bg="#ffffff", fg="#1a73e8", relief="groove", bd=2)
    form.pack(fill="x", padx=10, pady=10)
    form.columnconfigure(1, weight=1)

    self.app_id = make_form_row(form, 0, "Appointment ID:", make_entry)
    self.date = make_form_row(form, 1, "Date (YYYY-MM-DD):", make_entry)
    self.pat_id = make_form_row(form, 2, "Patient ID:", make_entry)
    self.doc_id = make_form_row(form, 3, "Doctor ID:", make_entry)
    self.start = make_form_row(form, 4, "Start Time (HH:MM:SS):", make_entry)
    self.end = make_form_row(form, 5, "End Time (HH:MM:SS):", make_entry)
    self.cost = make_form_row(form, 6, "Cost:", make_entry)

    tk.Label(form, text="Status:", font=("Segoe UI", 10), bg="#ffffff", anchor="w").grid(
        row=7, column=0, sticky="w", padx=(10, 5), pady=4)
    self.status = ttk.Combobox(form, values=["Scheduled", "In Progress", "Postponed"],
        font=("Segoe UI", 10), state="readonly")
    self.status.set("Scheduled")
    self.status.grid(row=7, column=1, sticky="ew", padx=(0, 10), pady=4)

    self.diagnosis = make_form_row(form, 8, "Diagnosis:", make_entry)

    btn_frame = tk.Frame(form, bg="#ffffff")
    btn_frame.grid(row=9, column=0, columnspan=2, pady=8)
    action_btn(btn_frame, "➕ Add", "#1a73e8", self.add).pack(side="left", padx=5)
    action_btn(btn_frame, "🗑 Delete", "#e53935", self.delete).pack(side="left", padx=5)
    action_btn(btn_frame, "🔄 Refresh", "#43a047", self.load).pack(side="left", padx=5)

    self.tree = build_tree(self, ["AppID", "Date", "Start", "End", "Cost", "Status", "Diagnosis", "PatientID", "DoctorID"])
    self.load()

def load(self):
    for r in self.tree.get_children(): self.tree.delete(r)
    try:
        con = get_connection(); cur = con.cursor()
        cur.execute("SELECT AppointmentID, AppointmentDate, StartTime, EndTime, Cost, Status, Diagnosis, PatientID, DoctorID FROM Appointment")
        for row in cur.fetchall(): self.tree.insert("", "end", values=row)
        con.close()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def add(self):
    try:
        con = get_connection(); cur = con.cursor()
        cur.execute("INSERT INTO Appointment (AppointmentID, AppointmentDate, StartTime, EndTime, Cost, Status, Diagnosis, PatientID, DoctorID) VALUES
        (%s,%s,%s,%s,%s,%s,%s,%s,%s)",
            (self.app_id.get(), self.date.get(), self.start.get(),
            self.end.get(), self.cost.get(), self.status.get(),
            self.diagnosis.get(), self.pat_id.get(), self.doc_id.get()))
        con.commit(); con.close()
        for f in [self.app_id, self.date, self.pat_id, self.doc_id,
            self.start, self.end, self.cost, self.diagnosis]:
            f.delete(0, "end")
        self.load()
        messagebox.showinfo("Success", "Appointment added!")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def delete(self):
    sel = self.tree.selection()
    if not sel: return messagebox.showwarning("Warning", "Select a row first!")
    val = self.tree.item(sel[0])["values"][0]
    if messagebox.askyesno("Confirm", f"Delete Appointment {val}?"):
        try:
            con = get_connection(); cur = con.cursor()
            cur.execute("DELETE FROM Appointment WHERE AppointmentID=%s", (val,))
            con.commit(); con.close(); self.load()
        except Exception as e:
            messagebox.showerror("Error", str(e))

# === Queries ===
class QueryTab(tk.Frame):
    def __init__(self, parent):
        super().__init__(parent, bg="#ffffff")
        self._build()

    def _build(self):
        top = tk.Frame(self, bg="#ffffff")
        top.pack(fill="x", padx=10, pady=10)

        self.queries = {
            "List patients diagnosed with fatty liver (last year)":

```

```

        "SELECT DISTINCT p.PatientID, p.PatientName FROM Patient p JOIN Appointment a ON p.PatientID=a.PatientID WHERE a.Diagnosis LIKE '%fatty
liver%' AND a.AppointmentDate >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)",
        "List addresses of cardiology clinics":
        "SELECT c.ClinicName, c.Address FROM Clinic c JOIN Department d ON c.DepartmentID=d.DepartmentID WHERE d.DepartmentName LIKE
'%cardiology%'",
        "Total money paid by patient ID 12527 (last 3 years)":
        "SELECT SUM(Cost) AS TotalPaid FROM Appointment WHERE PatientID=12527 AND AppointmentDate >= DATE_SUB(CURDATE(), INTERVAL 3
YEAR)",
        "All Doctors with their Department":
        "SELECT d.DoctorID, d.DoctorName, dep.DepartmentName FROM Doctor d JOIN Department dep ON d.DepartmentID=dep.DepartmentID",
        "Scheduled Appointments":
        "SELECT * FROM Appointment WHERE Status='Scheduled' ORDER BY AppointmentDate",
        "Custom SQL Query": ""
    }

    tk.Label(top, text="Select Query:", font=("Segoe UI", 11, "bold"),
            bg="#ffffff").pack(side="left", padx=(0, 10))
    self.combo = ttk.Combobox(top, values=list(self.queries.keys()),
            font=("Segoe UI", 10), width=45, state="readonly")
    self.combo.current(0)
    self.combo.pack(side="left")
    self.combo.bind("<<ComboboxSelected>>", self.on_select)
    action_btn(top, "> Run", "#1a73e8", self.run).pack(side="left", padx=10)

    self.sql_box = tk.Text(self, height=4, font=("Courier New", 10),
            bg="#f8f9fa", relief="solid", bd=1)
    self.sql_box.pack(fill="x", padx=10, pady=(0, 5))
    self.sql_box.insert("1.0", list(self.queries.values())[0])

    self.tree_frame = tk.Frame(self, bg="#ffffff")
    self.tree_frame.pack(fill="both", expand=True, padx=10, pady=5)

def on_select(self, event=None):
    key = self.combo.get()
    self.sql_box.delete("1.0", "end")
    self.sql_box.insert("1.0", self.queries[key])

def run(self):
    for w in self.tree_frame.winfo_children(): w.destroy()
    sql = self.sql_box.get("1.0", "end").strip()
    if not sql: return
    try:
        con = get_connection(); cur = con.cursor()
        cur.execute(sql)
        rows = cur.fetchall()
        cols = [d[0] for d in cur.description] if cur.description else []
        con.close()

    if not cols:
        messagebox.showinfo("Done", "Query executed successfully.")
        return

    tree = ttk.Treeview(self.tree_frame, columns=cols, show="headings")
    for c in cols:
        tree.heading(c, text=c); tree.column(c, width=140, anchor="center")
    sb = ttk.Scrollbar(self.tree_frame, orient="vertical", command=tree.yview)
    tree.configure(yscrollcommand=sb.set)
    tree.pack(side="left", fill="both", expand=True)
    sb.pack(side="right", fill="y")
    for row in rows: tree.insert("", "end", values=row)
    except Exception as e:
        messagebox.showerror("Query Error", str(e))

if __name__ == "__main__":
    app = ClinicApp()
    app.mainloop()

```

The results of the APP

Tables and forms as well as queries:



